

TRIVIA_TRAP

Team Roles & Execution Plan

5-person engineering team • Real-time multiplayer trivia & deception game

Recommended architecture

FastAPI • WebSockets • PostgreSQL • Redis • React • Docker

Purpose

Define clear ownership while keeping interfaces between the five subsystems explicit. Each developer owns a layer, but the team integrates continuously.

1. Team Overview

Developer	Role	Primary ownership
obouizi	Real-Time Engine Lead	WebSocket infrastructure, game state machine, timers, synchronization
eelkabia	Frontend SPA Lead	React SPA, game UI, client state, live-game experience
sjoukni	Game Rules & Data Lead	Database, trivia ingestion, validation, scoring and statistics
hakader	Authentication & Social Lead	OAuth, sessions, profiles, friends and chat
abougati	DevOps & Security Lead	Docker, Nginx, TLS, WAF, rate limiting, CI/CD and monitoring

2. obouizi — Real-Time Engine Lead

- Own FastAPI WebSocket infrastructure: connections, room subscriptions, broadcasting, authentication at connection time and connection lifecycle.
- Implement the server-authoritative game state machine: LOBBY → QUESTION → BLUFFING → VOTING → REVEAL → SCORING → FINISHED.
- Implement non-blocking asyncio timers and authoritative phase expiration.
- Handle join, leave, ready, start, answer submission, vote, reveal and state-update events.
- Implement disconnect detection, heartbeat/ping-pong and fast reconnect behavior.
- Prevent duplicate, late or invalid real-time actions at the protocol/state-machine boundary.
- Keep room state isolated so events are never broadcast to the wrong match.
- Coordinate with sjoukni for rule execution and with eelkabia for the WebSocket event contract.
- Bonus target: WebSocket Engine — 2.0

3. eelkabia — Frontend SPA Lead

- Own the React SPA architecture, routing, reusable components and client-side state management.
- Build lobby, room, player list, question, bluff, voting, reveal, scoreboard and final podium screens.
- Implement the WebSocket client and map server events into UI state.
- Build the live timer widget. The browser displays the server timer; it never decides when a phase ends.
- Build the answer/choice grid, submission states, voting states and reconnecting indicator.
- Implement responsive/mobile layouts and consistent interaction patterns.
- Integrate animation, audio and visual SFX hooks for important gameplay events.
- Coordinate with hakader on authentication/profile UI and with sjoukni on API contracts.
- Bonus target: SPA Framework — 1.5

4. sjoukni — Game Rules & Data Lead

- Own PostgreSQL, SQLAlchemy models, migrations, relationships, constraints and indexes.
- Implement question ingestion/API fetching, database seeding and question validation.

- Own the domain rules: bluff validation, voting rules, duplicate-answer handling and scoring.
- Implement profanity/sanitization rules for submitted bluffs.
- Implement exact-truth collision detection so a fake answer cannot equal the real answer.
- Implement automatic lie/answer timeout fallback when a player fails to submit in time.
- Implement the score engine: correct-answer points, deception points, successful traps and ties.
- Implement persistent match results and the deception statistics engine.
- Provide stable backend contracts to obouizi and eelkabia.
- Bonus target: ORM & Rules — 2.0

5. hakader — Authentication & Social Lead

- Implement 42 Intra OAuth 2.0 authentication flow and callback handling.
- Implement JWT/session handling, login/logout and protected user identity.
- Implement user profiles, avatars and account-related APIs.
- Implement player match history and profile statistics presentation.
- Implement friends: requests, accept/reject, remove, presence and blocking if required.
- Implement global, room and private chat features.
- Use obouizi's WebSocket infrastructure for chat rather than creating a second WebSocket system.
- Coordinate with abougati on security requirements and with eelkabia on authentication/social UI.
- Bonus target: OAuth & Chat — 2.0

6. abougati — DevOps & Security Lead

- Own Dockerfiles, Docker Compose, service networking, volumes, secrets and development/production environments.
- Configure Nginx as the reverse proxy for HTTP and WebSocket traffic.
- Configure HTTPS/TLS and secure transport.
- Implement ModSecurity/WAF rules and document the security policy.
- Implement rate limiting and abuse protection for authentication, API and sensitive endpoints.
- Set up health checks, readiness checks and basic monitoring.
- Own CI/CD: linting, tests, Docker builds and deployment workflow.
- Centralize logging and make failures diagnosable.
- Review security-sensitive changes from all developers.
- Keep infrastructure reproducible from a clean checkout.
- Bonus target: Security & WAF — 2.0

7. Critical Shared Contracts

These contracts must be designed together before dependent implementation begins.

Contract	Owners	Decision
WebSocket protocol	obouizi + eelkabia	Event names, payload schemas, errors, reconnect semantics
Game rules	obouizi + sjoukni	State transitions, valid actions, timeout behavior
Database/API	sjoukni + hakader	Models, IDs, auth data, persistence boundaries
Auth integration	hakader + eelkabia + abougati	OAuth callback, session/JWT handling, protected routes
Chat	hakader + obouizi + eelkabia	Chat event protocol, room scope, UI states
Deployment	abougati + all	Services, ports, environment, health checks

8. Development Workflow

Phase 1 — Foundation: abougati prepares Docker/CI; sjoukni creates DB foundations; eelkabia creates SPA shell; hakader prepares auth; obouizi creates the WebSocket skeleton.

Phase 2 — Vertical Slice: login → room → start → question → bluff → vote → reveal → score.

Phase 3 — Core Expansion: reconnects, validation, complete scoring, persistence and polished game UI.

Phase 4 — Social: profiles, friends, chat, invitations and match history.

Phase 5 — Quality & Bonuses: localization/RTL, accessibility, security, WAF, monitoring and E2E testing.

Phase 6 — Production: final integration, deployment, load testing, security review and documentation.

9. Architectural Boundary

obouizi owns how events move and when game states change. **sjoukni** owns what the rules mean and how results are calculated. **eelkabia** owns how the client displays authoritative results. **hakader** owns identity and social behavior. **abougati** owns the environment and security surrounding all of them.

Ownership does not mean isolation. Every major feature must be integrated through agreed API/WebSocket contracts and tested end-to-end.